

# C mputo Concurrente 2026

## Pr ctica 2: Exclusi n Mutua: Locks y Pools en Java

Alumna: Yessica Vianney Montes de Oca Aguilar

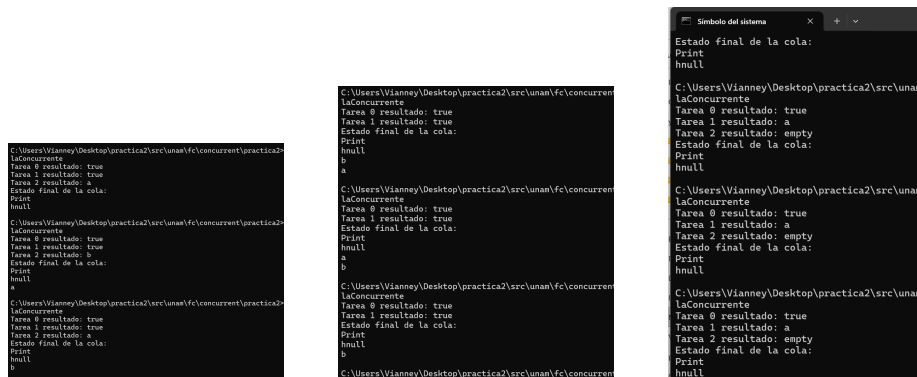
1. El programa ColaSecuencial es una implementaci n de una cola secuencial. Implementa una Cola concurrente utilizando una pool de hilos (ExecutorService). No utilices candados ni synchronized.

**Respuesta:**

ColaConcurrente.java:

Programa que implementa una cola concurrente a partir de la clase ColaSecuencial, utilizando un `ExecutorService` (pool de 4 hilos). Los m todos `enq(x)` y `deq()` se env an al pool como tareas (`Callable`) mediante `submit()`, sin usar candados ni `synchronized`, y los resultados se obtienen a trav s de los `Future` correspondientes.

2. Ejecuta varias veces tu implementaci n con diferentes secuencias de llamadas a m todos  Tu implementaci n funciona de acuerdo a lo que se espera de una cola o suceden inconsistencias? Por ejemplo: 1) Se hace `enq(a)`, `enq(b)` y `deq()` y al final ambos elementos a y b est n en la cola. 2) Se realiza un `enq(a)` y un `enq(b)` y la cola solo contiene al elemento a o b. 3) Se desencola dos veces el mismo elemento.



```
C:\Users\Vianney\Desktop\practica2\src\unam\fc\concurrent\practica2-
laConcurrente
Tarea 0 resultado: true
Tarea 1 resultado: true
Tarea 2 resultado: a
Estado final de la cola:
Print
null
b
a
C:\Users\Vianney\Desktop\practica2\src\unam\fc\concurrent\practica2-
laConcurrente
Tarea 0 resultado: true
Tarea 1 resultado: true
Estado final de la cola:
Print
null
a
b
C:\Users\Vianney\Desktop\practica2\src\unam\fc\concurrent\practica2-
laConcurrente
Tarea 0 resultado: true
Tarea 1 resultado: true
Tarea 2 resultado: a
Estado final de la cola:
Print
null
b
a
C:\Users\Vianney\Desktop\practica2\src\unam\fc\concurrent\practica2-
laConcurrente
Tarea 0 resultado: true
Tarea 1 resultado: true
Tarea 2 resultado: empty
Estado final de la cola:
Print
null
C:\Users\Vianney\Desktop\practica2\src\unam\fc\concurrent\practica2-
laConcurrente
Tarea 0 resultado: true
Tarea 1 resultado: a
Tarea 2 resultado: empty
Estado final de la cola:
Print
null
C:\Users\Vianney\Desktop\practica2\src\unam\fc\concurrent\practica2-
laConcurrente
Tarea 0 resultado: true
Tarea 1 resultado: a
Tarea 2 resultado: empty
Estado final de la cola:
Print
null
```

(a) Secuencia 1

(b) Secuencia 2

(c) Secuencia 3

Figura 1: Ejecuciones de las tres secuencias probadas en el Ejercicio 2.

**Respuesta:**

No, la implementación no funciona de manera consistente pues sí ocurren inconsistencias y varían dependiendo de la secuencia de llamadas utilizada.

**Secuencia enq(a), enq(b), deq()** (6 ejecuciones): solo 1 ejecución se comportó como se esperaría de una cola FIFO (**deq()** devolvió `.a`, quedando `b` en la cola), en las 5 restantes ocurrieron inconsistencias: en 3 de ellas **deq()** devolvió `b` en vez de `a` (viola el orden FIFO, aunque la cola quedó con un elemento) y en 2 de ellas la cola quedó completamente vacía al final, es decir, se perdió un elemento.

**Secuencia enq(a), enq(b)** (9 ejecuciones): corresponde al caso de ejemplo 2 del enunciado. De las 9, 4 terminaron correctamente con ambos elementos en la cola (`a`, `b`), 2 terminaron con ambos elementos pero en orden invertido (`b`, `a`), y en 3 se perdió un elemento por completo quedando la cola con un solo elemento en vez de dos.

**Secuencia enq(a), deq(), deq()** (8 ejecuciones): corresponde al caso de ejemplo 3 del enunciado. En estas 8 no se logró reproducir el caso de desencolar dos veces el mismo elemento; siempre se obtuvo `.a` en el primer **deq()** y `.empty` en el segundo. Esto no significa que la inconsistencia sea imposible en este caso, sino que la ventana de tiempo en la que los hilos compiten por `head` y `tail` es más angosta con solo tres tareas (un **enq** y dos **deq**), por lo que es menos probable que un hilo alcance a leer el estado de la lista a medio actualizar por otro hilo.

La cola concurrente sin candados ni **synchronized** no garantiza el comportamiento esperado de una cola: el resultado varía entre ejecuciones y depende del entrelazado (interleaving) de los hilos al acceder a la lista enlazada compartida.

3. ¿Existen data races o race-conditions? Explica en qué variables y por qué suceden.

**Respuesta:**

Sí, existen ambas, sobre los mismos campos: `head.next`, `tail.next` y el campo `next` de los nodos de la lista enlazada ya que `ColaSecuencial` no los protege con ningún candado ni **synchronized**.

**Data race:** en **enq()** se lee y escribe `head.next` (o `tail.next`) para insertar un nodo y en **deq()** se lee y escribe `head.next` para extraer el primero; como mínimo un acceso es de escritura y ocurren al mismo tiempo sobre el mismo campo, hay data race.

**Race condition:** si dos **enq()** concurrentes evalúan `head.next == tail` al mismo tiempo cuando la cola está vacía, ambos pueden entrar al mismo bloque y uno sobrescribe el `head.next` del otro, perdiendo un elemento. Igual, si dos **enq()** leen `last = tail.next` antes de que se actualice, ambos escriben sobre el mismo nodo y se pierde el elemento intermedio.

4. Utiliza candados y/o `synchronized` en tu implementación de forma que los métodos `enq()` y `deq()` sean una sola sección crítica y contesta: ¿existen inconsistencias?

**Respuesta:**

`ColaConcurrenteSync.java`

Programa que reutiliza `ColaSecuencial` igual que en el Ejercicio 1, pero envuelve las llamadas a `enq()` y `deq()` dentro de un bloque `synchronized` sobre el mismo objeto (`cola`). Como ambos métodos sincronizan sobre el mismo monitor, forman una sola sección crítica: mientras un hilo está ejecutando cualquiera de los dos, ningún otro hilo puede entrar al otro método al mismo tiempo.

Se probó la misma secuencia `enq(a)`, `enq(b)`, `deq()` del Ejercicio 2 en 9 ejecuciones. A diferencia de la versión sin protección, aquí no hubo pérdida de elementos ni la cola quedó vacía o con ambos elementos: en las 9 ejecuciones la cola terminó con exactamente un elemento, como se espera. Lo único que varió fue cuál elemento devolvió `deq()` (5 veces `.a`, 4 veces `.b`), pero esto no es una inconsistencia de la estructura de datos, sino consecuencia de que el orden en que se envían las tareas con `submit()` no garantiza el orden en que el pool las ejecuta; en algunas ejecuciones `deq()` corrió antes de que `enq(a)` terminara, encontrando solo `.b` disponible.

Con `synchronized` ya no existen inconsistencias en la estructura de la cola (no se pierden ni duplican elementos), aunque el orden exacto de las operaciones sigue sin ser determinístico por tratarse de tareas asíncronas independientes.

5. En una pool de hilos (`ExecutorService`), ¿si utilizamos más hilos equivale a un mayor throughput? Argumenta porqué.

**Respuesta:**

No necesariamente. Aumentar el número de hilos ayuda a mejorar el throughput solo hasta cierto punto, después el rendimiento se estanca o incluso empeora, por varias razones:

- **Núcleos físicos limitados:** una máquina solo puede ejecutar en verdadero paralelo tantos hilos como núcleos de CPU tenga; más allá de eso, los hilos adicionales solo se reparten el mismo tiempo de procesador mediante *context switching*.
- **Costo del cambio de contexto:** cada vez que el sistema operativo cambia de un hilo a otro hay un costo (guardar y restaurar registros, mapeo de memoria, etc.); con demasiados hilos, este costo puede llegar a consumir más tiempo que el trabajo útil realizado.
- **Contención por recursos compartidos:** si las tareas usan secciones críticas (como en los Ejercicios 1 y 4), más hilos compitiendo por el mismo candado no aceleran el trabajo, solo aumentan la espera.

- **Naturaleza de la tarea:** si las tareas son intensivas en CPU (*CPU-bound*), el número óptimo de hilos ronda el número de núcleos disponibles; si son tareas que esperan mucho en I/O (*I/O-bound*), sí conviene tener más hilos que núcleos, porque mientras unos esperan, otros pueden usar la CPU.

El throughput crece con el número de hilos solo hasta un punto óptimo relacionado con los núcleos disponibles y el tipo de tarea; pasado ese punto, agregar más hilos degrada el rendimiento en vez de mejorarlo.

6. Problema. Supón que perteneces a un equipo de trabajo en el cual estás a cargo de dar acceso a un servidor. Seis personas te pueden mandar tareas para que las ejecutes en el servidor. Las tareas del hilo 0 y 2 tardan 500ns, las del hilo 1 tardan 2000ns y las de los hilos 3, 4 y 5 tardan 3000ns. Tienes la instrucción de no dar acceso a más de tres al mismo tiempo y de no aceptar al mismo tiempo las tareas del hilo 0 y 2. Diseña e implementa una solución. ¿Tu implementación cumple con Justicia? Si no es así, describe cómo podrías garantizarla.

**Respuesta:**

Se modificaron los programas `Tarea.java` y `Scheduler.java`. En `Tarea`, cada tarea primero adquiere el `Semaphore` (inicializado en 3 en `Scheduler`) antes de ejecutar su trabajo y lo libera al terminar, garantizando que nunca haya más de 3 tareas corriendo al mismo tiempo. Además, se agregó un `Lock` estático compartido únicamente entre las tareas del hilo 0 y del hilo 2 (identificados por `value == 0 || value == 2`); solo cuando una tarea pertenece a uno de estos dos hilos adquiere ese candado antes de ejecutar y lo libera después, de forma que las tareas 0 y 2 nunca corren traslapadas entre sí, sin afectar en nada a los hilos 1, 3, 4 y 5.

Se probó la implementación con las 26 tareas del `Scheduler` original, verificando en la traza de ejecución que en ningún momento hubo más de 3 tareas activas simultáneamente, y que ninguna ejecución del hilo 0 se traslapó con una del hilo 2.

**¿Cumple con Justicia?** No. Tanto `Semaphore` como `ReentrantLock` en Java, por defecto, no garantizan un orden FIFO al decidir qué hilo en espera obtiene el permiso o el candado cuando este se libera; usan un esquema no justo (*non-fair*) que puede favorecer a unos hilos sobre otros de forma no determinista. Esto significa que, en teoría, una tarea podría quedar esperando indefinidamente si el sistema sigue favoreciendo a otras.

Para garantizar justicia, ambas clases permiten construirse en modo "fair", pasando `true` como parámetro adicional: `new Semaphore(3, true)` y `new ReentrantLock(true)`. En este modo, los hilos obtienen el permiso o el candado en el mismo orden en que lo solicitaron (orden FIFO de espera), evitando la inanición (*starvation*) de cualquier tarea.